# GraphDBManager

## System Design Description

Eclipse Arrowhead 5.2.2 | support system | Revision 2026-11-09 | REVIEW DRAFT

## Purpose and document structure

This document describes implementation ownership, architecture, persistence, security and operational boundaries. The current cross-language sections below precede the detailed implementation appendices. Appendix dates and release labels belong to the retained baseline, not to a newly qualified release. Existing descriptions have been assembled from the primary trees; no live interoperability or semantic-parity claim is made.

## Java implementation

Primary folder: Eclipse-Arrowhead-java/ah5-GraphDBManager-java-spring. 1 detailed Java SysDD source document(s) are included in the appendices. Their architecture, operation parameters, models, transport bindings, access rules and errors remain associated with Java. They must not be read as automatically implemented by the ports.

## Go implementation

No Go implementation folder for GraphDBManager was found in the designated primary Go tree on 2026-11-09. No executable, build command, endpoint or feature parity is asserted. A future port must define architecture, state migration, service registration, DTO validation, authorization and transport coverage before it can replace this system.

## Rust implementation

No Rust implementation folder for GraphDBManager was found in the designated primary Rust tree on 2026-11-09. No executable, build command, endpoint or feature parity is asserted. A future port must define architecture, state migration, service registration, DTO validation, authorization and transport coverage before it can replace this system.

## Cross-language qualification

Identical system names or HTTP paths do not establish interoperability. For the selected implementation, verify registered service versions and metadata, method/path and payload models, null and optional-field handling, success/error bodies, authentication and per-operation authorization, timeout/retry behavior and persistence across restart. Java MQTT/AMQP descriptions apply to a port only where its own implementation evidence explicitly supports them. Missing transports must not be advertised. State conversion from Java SQL to native snapshots, secret provisioning, backup/restore and rollback require implementation-specific procedures. No data migration, deployment or cloud reconfiguration was performed for this document set.

## Detailed appendices and provenance

- Java: Eclipse-Arrowhead-java/ah5-GraphDBManager-java-spring/documentation/v5.2.2/GraphDBManager-v5.2.2-SysDD.tex

The package sources/ folder retains the selected LaTeX, figures, bibliography, language README/build evidence and source snapshots for newly described contracts. source-manifest.json records SHA-256 hashes of original inputs. build-report.json records whether an appendix used an existing newer PDF or was compiled from source.

# GraphDBManager

## System Design Description

**Abstract**
This document describes the design and implementation of the GraphDBManager support system of Arrowhead 5.2.2 (`ah5-GraphDBManager-java-spring`).

# Contents

**1 Overview** **3**

**2 Implementation** **4**
2.1 Implementation language and tools . . . . . 4
2.2 Independent source and build ownership . . . . . 4
2.3 Functional properties implementation . . . . . 4
2.4 Non functional properties implementation . . . . . 5
2.5 Deployment . . . . . 6
2.6 Local modifications . . . . . 6
2.7 Verification . . . . . 6

**3 Services** **7**

**4 References** **7**

**5 Revision History** **9**
5.1 Amendments . . . . . 9
5.2 Quality Assurance . . . . . 9

# 1 Overview

This document describes the GraphDBManager system implementation, which realizes the GraphDBManager SysD [1]. GraphDBManager caches and persists SenML (RFC 8428) sensor and actuator data: `graphDataProxy` keeps the latest message per system and service, `graphDataHistorian` the time series, kept as RDF in Ontotext GraphDB [2]. In Section 2, we describe implementation details of the system.

# 2 Implementation

This implementation is based on the SysD document GraphDBManager [1] and is located in `ah5-GraphDBManager-java-spring`.

## 2.1 Implementation language and tools

- Java 21, Spring Boot 3.4.9, Arrowhead common library 5.2.2 (private source under `vendor/`), Log4j 2; Eclipse RDF4J 5.1.2 client [3] (`rdf4j-client`, HTTP repository).
- Ontotext GraphDB Free 10.8.5 [2] as RDF store: one repository `graphdbmanager` (GraphDB-Free sail, `empty` ruleset, context index on). No relational database, JPA or Hibernate. Stateless request processing apart from the in-memory proxy cache.

## 2.2 Independent source and build ownership

The project folder `ah5-GraphDBManager-java-spring` is a copy of the standalone DataManager folder [4] and owns its application source, Maven build, tests, executable JAR, Dockerfiles (application and GraphDB), configuration and Compose definition. Build with Java 21 and `mvn -B clean verify`; after a first build that fetches RDF4J from Maven Central, offline builds work with the populated cache.

Arrowhead common utilities and DTOs are private source copies in `vendor/`, compiled into this system by the local Maven POM. In this copy the JPA-dependent parts of the common library are removed (`common/jpa`, `LogService`, `LogValidation`, the Log4j 2 JDBC connection factory); `Constants` holds the former entity column sizes. The removals are recorded in `vendor/PROVENANCE.json`.

The packaged artifact is `arrowhead-graphdbmanager-5.2.2.jar` (main class `eu.arrowhead.graphdbmanager.GraphDBManagerMain`).

## 2.3 Functional properties implementation

The following components (packages below `eu.arrowhead.graphdbmanager`) implement the system:

| Component | Responsibility |
|---|---|
| `api.http` | Controllers of graphDataProxy and graphDataHistorian. |
| `service` | Proxy cache and historian logic, SenML validation, name normalization, sysMonitor health indicator. |
| `security` | Operation resolver and authorization filter (ConsumerAuthorization `verify` per call). |
| `graphdb` | `HistorianStore` interface, `HistorianPack` record and `GraphDBHistorianStore` (RDF4J `HTTPRepository`). |
| `init` | Re-registration of the HTTP offers with `CERT_AUTH`. |

Every request passes the common filter chain: authentication filter (`authentication.policy`), optional blacklist filter, management filter (management services), the system's authorization filter, then the controller, validation and normalization, the service logic and the historian store.

### 2.3.1 Data handled and stored

`put-data` on graphDataHistorian writes one SenML pack in one RDF4J transaction into the named graph `<urn:arrowhead:graphdbmanager:historian>` of the repository. Vocabulary prefix `gdm: = https://arrowhead.eu/ns/graphdbmanager#`:

| Resource / property | Content |
|---|---|
| <urn:uuid:...> a gdm:SenMLPack | One stored pack (random UUID). |
| gdm:systemName, gdm:serviceName | Normalized endpoint (string literals). |
| gdm:baseTime, gdm:minTime, gdm:maxTime | Base time and time range of the pack, xsd:double seconds; used by the time-range query. |
| gdm:payload | The normalized pack as SenML JSON, returned unchanged by get-data. |
| gdm:createdAt | Storage time, xsd:dateTime. |
| gdm:measurement → a gdm:Measurement | One resource per named record (<urn:uuid:...#m*i*>) with gdm:name, gdm:unit (record or base unit), absolute gdm:time and gdm:value, gdm:stringValue or gdm:booleanValue; for SPARQL users. |

list-systems, list-services and get-data are SPARQL SELECT queries [5] on that graph (DISTINCT names; packs with maxTime >= from and minTime <= to, newest base time first, LIMIT count). Request values are bound to query variables as RDF literals, never concatenated into the query text. graphDataProxy keeps the last pack per endpoint in memory only.

The data models of the interfaces are Java records (of eu.arrowhead.dto in the common library, or of the module); the IDD documents name them without the DTO suffix. The records used by this system are: SenMLRecordDTO, ServiceListResponseDTO, SystemListResponseDTO.

## 2.4 Non functional properties implementation

### 2.4.1 Security

TLS and client authentication are configured with the Spring server.ssl.* properties (PKCS#12 keystore and truststore, client-auth=need). The common library's filters implement the authentication policies; Token CertificateBinding binds identity tokens to the client certificate and CrlRevocationConfig enables CRL checking. GraphDBManager authenticates to GraphDB with HTTP Basic as user graphdbmanager, which has only READ_REPO_graphdbmanager and WRITE_REPO_graphdbmanager.

### 2.4.2 Power management

None; the system is a server process without power states. The sysMonitor power observation always reports line supply.

### 2.4.3 Internal monitoring

Log4j 2 logs to stdout and files; there is no database log appender, so generalManagement.get-log has no stored entries. The common library's SysMonitorPublisher publishes echo, power and status every 30 s; echo is degraded while the historian query fails.

### 2.4.4 Configuration

Configuration is read from application.properties and environment variables (Spring relaxed binding, e.g. GRAPHDB_URL); secrets come from files through SPRING_CONFIG_IMPORT=configtree:/run/secrets/ (ah.graphdb-password, ah.store-password, ah.identity-password). The properties are listed in the SysD; generalManagement.get-config returns the public ones.

## 2.5 Deployment

Two containers: the application (UID 10001, read-only root filesystem, no capabilities, HTTPS port 8467) and `graphdbmanager-graphdb`, built from `ontotext/graphdb:10.8.5` by `deployment/graphdb/Dockerfile` (UID 10001, read-only root filesystem, data volume `/opt/graphdb/home`, no host port). Its `entrypoint.sh` starts GraphDB and on every start, idempotently, creates the repository from `repository-config.ttl`, sets the admin password, creates or updates the `graphdbmanager` user, enables GraphDB security and disables free access; only then is the container healthy. Passwords are read from files and passed to curl through a temporary netrc file and standard input. The two containers share an internal network that no other system joins; GraphDB is reached over plain HTTP on it.

The local `compose.yaml` runs both on an operator-selected external cloud network (`docker compose up -d -build`). The Arrowhead-test-cloud-java test cloud is a separate deployment consumer of this primary source: there the services `graphdbmanager` and `graphdbmanager-graphdb` are added by `make up-graphdbmanager`: certificate and keystore (`generate-x509.sh`), identity (`provision-authentication-identities.sh`), AMQP access list, ConsumerAuthorization grants (`configure-graphdbmanager-policies.sh`, verifier and sysMonitor provider entries), then the two containers; the internal network `db-graphdbmanager` has a fixed subnet. ServiceRegistry, Authentication, DSO, ConsumerAuthorization and the AMQP broker remain network dependencies.

## 2.6 Local modifications

Notes on this implementation in the workspace:

- Copy of DataManager 5.2.2 (2026-10-28) with JPA/MySQL replaced by GraphDB; DataManager's `/echo` replacement by `sysMonitor` is kept.
- Service definitions `graphDataProxy` and `graphDataHistorian`, base path `/graphdbmanager`: ServiceRegistry's reserved service definitions bind `dataProxy` and `dataHistorian` to DataManager.
- `put-data` answers 204.

## 2.7 Verification

The JUnit suite covers the historian service with a mocked store, SenML validation, the proxy, the health indicator and the operation resolver and authorization filter. In the Arrowhead-test-cloud-java deployment consumer, `scripts/smoke-graphdbmanager.sh` checks, as SmokeClient and through orchestration only, for both HTTP services: `put-data`, `get-data` round trip and `list-systems`; then a SPARQL count of the stored measurement in GraphDB and the refusal (401) of anonymous GraphDB access. `smoke-test.sh` checks mTLS and identity-token access and `smoke-security.sh` the port's certificate and CRL handling. Historian data survived a restart of the GraphDB container.

# 3 Services

The implementation of the services is based on the SD and IDD documents referenced below.

| **Services produced** | **SysD ref** | **SD ref** | **IDD ref** |
|---|---|---|---|
| `graphDataProxy` | [1] | [6] | [7] |
| `graphDataHistorian` | [1] | [8] | [9] |
| `sysMonitor` | [1] | [10] | [11] |
| | | | |
| **Services consumed** | **SysD ref** | **SD ref** | **IDD ref** |
| systemDiscovery, serviceDiscovery | [12] | [13] | – |
| identity | [14] | [15] | – |
| authorization | [16] | [17] | – |
| authorization | [16] | [17] | – |
| sysMonitor (publisher) | AMQP broker (SysMonitor) | – | – |
| RDF4J HTTP protocol (GraphDB) | – | – | [3, 2] |

# 4 References

[1] J. Delsing, “GraphDBManager – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-GraphDBManager-java-spring/documentation/v5.2.2/GraphDBManager-v5.2.2-SysD.pdf.

[2] Ontotext, “GraphDB 10.8 Documentation,” https://graphdb.ontotext.com/documentation/10.8/, 2025.

[3] Eclipse Foundation, “Eclipse RDF4J 5,” https://rdf4j.org/, 2024.

[4] J. Delsing, “DataManager – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-DataManager-java-spring/documentation/v5.2.2/DataManager-v5.2.2-SysD.pdf.

[5] S. Harris and A. Seaborne, “SPARQL 1.1 Query Language,” W3C Recommendation, 2013.

[6] J. Delsing, “graphDataProxy – Service Description (GraphDBManager),” Eclipse Arrowhead, SD 5.2.2, 2026, graphDBManager-v5.2.2-graphDataProxy-SD.pdf.

[7] ——, “graphDataProxy HTTP/TLS/JSON – Interface Design Description (GraphDBManager),” Eclipse Arrowhead, IDD 5.2.2, 2026, graphDBManager-v5.2.2-graphDataProxy-HTTP-IDD.pdf.

[8] ——, “graphDataHistorian – Service Description (GraphDBManager),” Eclipse Arrowhead, SD 5.2.2, 2026, graphDBManager-v5.2.2-graphDataHistorian-SD.pdf.

[9] ——, “graphDataHistorian HTTP/TLS/JSON – Interface Design Description (GraphDBManager),” Eclipse Arrowhead, IDD 5.2.2, 2026, graphDBManager-v5.2.2-graphDataHistorian-HTTP-IDD.pdf.

[10] ——, “sysMonitor – Service Description (GraphDBManager),” Eclipse Arrowhead, SD 5.2.2, 2026, graphDBManager-v5.2.2-sysMonitor-SD.pdf.

[11] ——, “sysMonitor AMQP/TLS/JSON – Interface Design Description (GraphDBManager),” Eclipse Arrowhead, IDD 5.2.2, 2026, graphDBManager-v5.2.2-sysMonitor-AMQP-IDD.pdf.

[12] ——, “ServiceRegistry – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/serviceregistry/documentation/v5.2.2/ServiceRegistry-v5.2.2-SysD.pdf.

[13] ——, “systemDiscovery – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.2.2, 2026, ah5-core-java-spring/serviceregistry/documentation/v5.2.2/ServiceRegistry-v5.2.2-systemDiscovery-SD.pdf.

[14] ——, “Authentication – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/authentication/documentation/v5.2.2/Authentication-v5.2.2-SysD.pdf.

[15] ——, “identity – Service Description (Authentication),” Eclipse Arrowhead, SD 5.2.2, 2026, ah5-core-java-spring/authentication/documentation/v5.2.2/Authentication-v5.2.2-identity-SD.pdf.

[16] ——, “ConsumerAuthorization – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/consumer-authorization/documentation/v5.2.2/ConsumerAuthorization-v5.2.2-SysD.pdf.

[17] ——, “authorization – Service Description (ConsumerAuthorization),” Eclipse Arrowhead, SD 5.2.2, 2026, ah5-core-java-spring/consumer-authorization/documentation/v5.2.2/ConsumerAuthorization-v5.2.2-authorization-SD.pdf.

# 5 Revision History

## 5.1 Amendments

| No. | Date | Version | Subject of Amendments | Author |
|---|---|---|---|---|
| 1 | 2026-10-28 | 5.2.2 | Document created from the DataManager SysDD for the GraphDB-backed copy GraphDBManager. | Jerker Delsing |
| 2 | 2026-10-30 | 5.2.2 | Checked against the primary Java source tree (Eclipse-Arrowhead-java): services, operations, paths and data models unchanged. | Claude Opus 5.5 (draft) |

## 5.2 Quality Assurance

| No. | Date | Version | Approved by |
|---|---|---|---|
| 1 | | 5.2.2 | |